



НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ

«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ
СІКОРСЬКОГО»

ФАКУЛЬТЕТ ПРИКЛАДНОЇ МАТЕМАТИКИ

**Кафедра системного програмування та спеціалізованих
комп'ютерних систем**

Лабораторна робота №3

з дисципліни **Web-дизайн**

на тему: “Розробка Web-додатка засобами Javascript/React”

Виконав: студент III курсу

ФПМ групи КВ-23

Зінедін Шайдін

Перевірив:

Мета роботи: ознайомитись із засобами фреймворка React та навчитись створювати асинхронні запити до Web-сервера.

Загальне завдання: адаптувати програмний код ЛР№2 до вимог фреймворка React та забезпечити завантаження необхідних даних з Web-сервера.

Інструменти розробки: Javascript, Vue.js, SQLite, Axios, Bootstrap, Vue CLI, Webpack, NodeJS, Express.

Посилання на Github-репозиторій: https://github.com/9koto/Lab3_WEB/

Контакт студента у месенджері Телеграм: @ninekoto

Опис додатка «Облік робочого часу»

Додаток передбачає 5 сторінок:

1. Реєстрація користувача (поля: ім'я, email, пароль, дата народження);
2. Вхід до сайту (поля: email, пароль);
3. Профіль користувача (поля у табличному вигляді);
4. Про додаток (короткий опис додатка);
5. Робоча сторінка додатка на якій знаходяться кнопки для запуску, паузи і зупинки таймера, а також циферблат;

Список Vue-компонентів та схема їх взаємодії

- **App:** головний компонент додатка, який містить навігаційне меню та точку монтування для маршрутизації.
- **Login:** компонент сторінки входу. Виконує авторизацію за email та паролем.
- **Register:** компонент сторінки реєстрації. Дозволяє створювати нового користувача.
- **Profile:** компонент сторінки профілю. Відображає інформацію про користувача.
- **About:** компонент сторінки «Про додаток». Містить опис додатка та емблему.
- **Dashboard:** компонент робочої сторінки. Відображає таймер, кнопки управління та список сесій.

Схема взаємодії: компонент App використовує Vue Router для маршрутизації між сторінками. Кожен компонент взаємодіє з сервером через Axios запити (наприклад, /api/register, /api/login, /api/sessions). Дані користувача зберігаються у localStorage, а сесії — у базі даних SQLite.

Копії екрана робочої сторінки



Облік робочого часу

Назва сеансу

Study

Час: 118 сек

Старт

Пауза

Стоп

Код компоненту робочої сторінки

```
<template>
  <div>
    <h1 class="text-center mb-4">Облік робочого часу</h1>
    <div class="w-50 mx-auto">
      <div class="mb-3">
        <label for="session-name" class="form-label">Назва сеансу</label>
        <input v-model="sessionName" type="text" class="form-control"
id="session-name" placeholder="Введіть назву сеансу">
      </div>
      <div class="mb-3 text-center">
        <h3>{{ timerDisplay }}</h3>
      </div>
      <div class="text-center mb-3">
        <button @click="startTimer" class="btn btn-success me-2">Старт</button>
        <button @click="pauseTimer" class="btn btn-warning me-2">Пауза</button>
        <button @click="stopTimer" class="btn btn-danger">Стоп</button>
      </div>
      <h4>Список сеансів</h4>
      <ul class="list-group">
        <li v-for="session in sessions" :key="session.id" class="list-group-
item">
          Назва: {{ session.name }}, Початок: {{ session.start }}, Кінець: {{
session.end }}, Тривалість: {{ session.duration }} сек
        </li>
      </ul>
    </div>
  </div>
</template>

<script>
import axios from 'axios';

export default {
  data() {
    return {
      sessionName: '',
      startTime: null,
      timerInterval: null,
      elapsedTime: 0,
      sessions: [],
      user: JSON.parse(localStorage.getItem('currentUser')) || {},
    };
  },
  computed: {
    timerDisplay() {
      return `Час: ${this.elapsedTime} сек`;
    },
  },
}
```

```

    },
    methods: {
      async startTimer() {
        if (!this.timerInterval) {
          this.startTime = new Date();
          this.timerInterval = setInterval(() => {
            const now = new Date();
            this.elapsedTime = Math.floor((now - this.startTime) / 1000);
          }, 1000);
        }
      },
      pauseTimer() {
        clearInterval(this.timerInterval);
        this.timerInterval = null;
      },
      async stopTimer() {
        clearInterval(this.timerInterval);
        this.timerInterval = null;
        const endTime = new Date();
        const duration = Math.floor((endTime - this.startTime) / 1000);
        const session = {
          userId: this.user.id,
          name: this.sessionName || 'Сессия',
          start: this.startTime.toLocaleString(),
          end: endTime.toLocaleString(),
          duration,
        };
        await axios.post('http://localhost:3000/api/sessions', session);
        this.fetchSessions();
        this.elapsedTime = 0;
      },
      async fetchSessions() {
        const response = await
axios.get(`http://localhost:3000/api/sessions/${this.user.id}`);
        this.sessions = response.data;
      },
    },
    mounted() {
      this.fetchSessions();
    },
  };
</script>

```